

Documentazione del Progetto: Programmazione Object Oriented

TAMAGOTCHI

Sistema di Gestione per Videogioco

Zazzaro Flavia (DE1000158)
Università degli Studi di Napoli Federico II

Zinno Raquel (DE1000012)
Università degli Studi di Napoli Federico II

Indice

1	Specifica dei Requisiti	3
1.1	Scopo e ambito del sistema	3
1.2	Requisiti funzionali (Casi d'Uso)	3
1.3	Requisiti di Progettazione Object-Oriented	4
2	Diagramma delle Classi e presentazione del dominio	5
2.1	Class Diagram	5
2.2	Classi del Dominio	5
2.3	Associazioni e Cardinalità	7
2.4	Regole di business	7
3	Lista delle funzioni ad alto livello	8
3.1	Gestione Utente	8
3.2	Gestione Animale	9
3.3	Gestione Cibo	10
3.4	Gestione Vestito	11
3.5	Gestione Minigame	12
4	Architettura del software e Struttura dei package	13
4.1	BCE + DAO Pattern	13
4.2	Singleton Pattern per la Persistenza	13
4.3	Incapsulamento e Disaccoppiamento	14
5	Gestione delle eccezioni	15
5.1	Gestione su più livelli	15
5.2	Eccezioni personalizzate	15
6	Sequence diagram delle operazioni critiche	16
6.1	Operazione 1: creaAnimale	16
6.2	Operazione 2: compra	17
7	Repository Github	17

1. SPECIFICA DEI REQUISITI

1.1 Scopo e ambito del sistema

Il sistema permette a un utente di creare uno o più animali virtuali (fino a un massimo di 2 per ogni utente) scegliendo fra due diverse tipologie, un orso e un pinguino. L'utente avrà uno username e una password associati e sarà capace, dopo l'accesso iniziale, di gestire la creazione e l'eliminazione degli animali, scegliere l'animale con cui giocare fra quelli creati, comprare degli item, dare da mangiare e vestire l'animale.

Ogni animale ha un nome scelto e modificabile dall'utente, un livello di energia e un livello di fame gestiti come interi. Inoltre, ognuno dei due tipi di animale gestisce i propri livelli in maniera differente. Col passare del tempo i livelli di fame ed energia si abbasseranno. Per riguadagnare l'energia l'animale può dormire, per poi svegliarsi successivamente. Invece per riguadagnare la fame l'utente può dargli da mangiare del cibo.

Tramite il sistema sarà possibile giocare a dei minigame, ognuno dotato di nome, ricompensa in punti ed energia consumata: in caso di vittoria, l'animale perderà energia ma guadagnerà dei punti; in caso di sconfitta perderà solamente energia.

I punti possono essere utilizzati dall'utente per comprare gli item. Questi hanno un costo e un nome, in più sono suddivisi in vestiti e cibo: del cibo bisogna conoscere il valore di punti fame che ricarica quando utilizzato e il tipo (salato, dolce o bevanda); invece i vestiti possono essere indossati dall'animale e possiedono dei valori di boost, poiché quando usati aumenteranno il valore massimo dei livelli di fame/energia.

1.2 Requisiti funzionali (Casi d'Uso)

- Crea account
- Accede all'account
- Cambia password
- Crea l'animale
- Esce dall'account
- Seleziona l'animale
- Elimina animale
- Cambia nome animale
- Mette a dormire/sveglia l'animale
- Usa Item/rimuove vestito
- Acquista Item
- Elimina Item
- Gioca ai minigame

1.3 Requisiti di Progettazione Object-Oriented

- **Incapsulamento:** la configurazione degli attributi delle classi è stata ideata sfruttando diversi livelli appositi di visibilità (private, protected, public). In modo che questi ultimi possano essere letti o modificati esclusivamente tramite l'utilizzo degli appositi metodi getter e setter.
- **Ereditarietà:** Creazione di una classe astratta Animale contenente, che estende le classi Orso e Pinguino, e una classe astratta Item che estende le classi Cibo e Vestito.
- **Affidabilità:** il programma sarà capace di gestire qualsiasi eccezione relativa al database o al programma, evitando la perdita di dati dell'utente.
- **Architettura software:** il codice sorgente verrà strutturato seguendo il pattern architetturale BCE + DAO.
- **Separazione dei ruoli:** il sistema verrà sviluppato dividendo il codice in package diversi, in particolare la parte dedicata all'interfaccia grafica (gui), la logica di controllo (controller), i dati (model), e il codice database (dao).

2. DIAGRAMMA DELLE CLASSI E PRESENTAZIONE DEL DOMINIO

2.1 Class Diagram

Lo schema concettuale rappresenta il dominio applicativo in formato astratto.

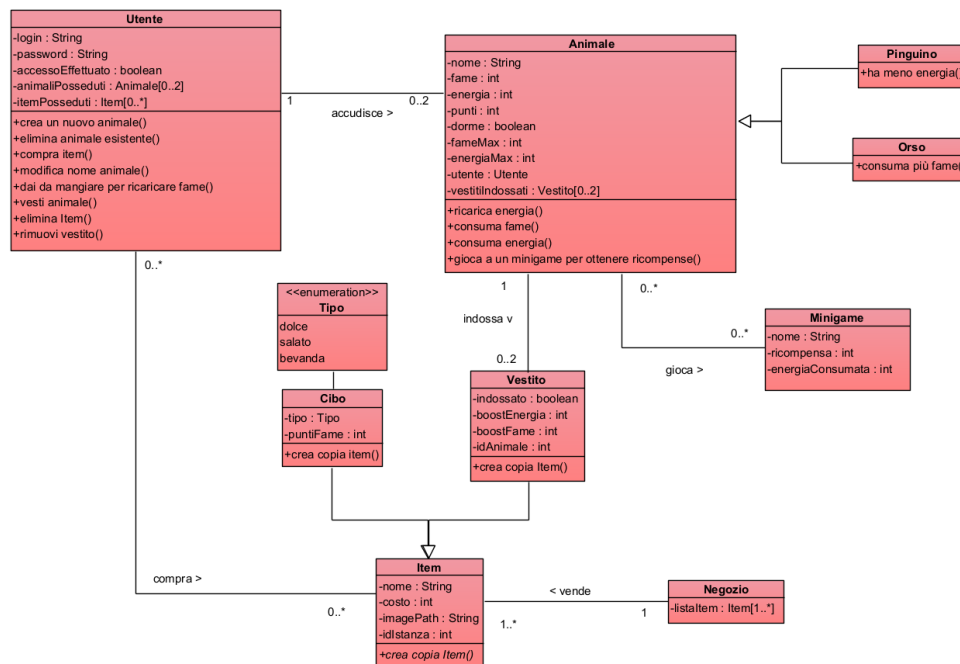


Figura 1: Diagramma delle classi UML del sistema Tamagotchi.

2.2 Classi del Dominio

Nel package model sono state tradotte le classi dallo schema concettuale, rappresentando le entità del sistema:

1. **Utente:** Rappresenta una persona registrata nel database del sistema.
 - **Attributi:** login, password, accessoEffettuato, animaliPosseduti, itemPosseduti.
 - **Metodi:** creaAnimale(), eliminaAnimale(), compraItem(), daiCibo(), vesti(), eliminaItem(), rimuoviVestito().
2. **Animale:** Animale posseduto da un utente.
 - **Attributi:** nome, fame, fameMax, energia, energiaMax, punti, dorme, utente, vestitiIndossati.
 - **Metodi:** caricaEnergia(), consumaFame(), consumaEnergia(), gioca().

3. **Orso (Sottoclasse di Animale):** Tipo di animale.
 - **Metodi:** toString() per stampare nome dell'animale, consumaFame() per consumare la fame dell'animale più velocemente.
4. **Pinguino (Sottoclasse di Animale):** Tipo di animale.
 - **Metodi:** toString() per stampare nome dell'animale.
5. **Item (Classe Astratta):** Rappresenta gli oggetti acquistabili nel negozio e utilizzabili dall'utente.
 - **Attributi:** nome, costo, negozio, iconPath, idIstanza.
 - **Metodi:** creaCopia() (metodo astratto) crea una copia dell'oggetto per inserirla nell'inventario dell'animale.
6. **Cibo (Sottoclasse di Item):** Cibo acquistabile nel negozio.
 - **Attributi:** tipo, puntiFame.
 - **Metodi:** toString() per stampare nome e attributi del cibo, creaCopia() per creare una copia del cibo.
7. **Vestito (Sottoclasse di Item):** Vestito acquistabile nel negozio.
 - **Attributi:** indossato, boostEnergia, boostFame, idAnimale.
 - **Metodi:** toString() per stampare nome e attributi del vestito, creaCopia() per creare una copia del vestito.
8. **Negozi:** Rappresenta il negozio per comprare gli items.
 - **Attributi:** listaItem.
 - **Metodi:** getListaItem().
9. **Minigame:** Rappresenta i minigames possibili da giocare.
 - **Attributi:** nome, ricompensa, energiaConsumata.
 - **Metodi:** getter degli attributi corrispondenti.
10. **TipoCibo:** Enumerazione per il cibo.
 - salato
 - dolce
 - bevanda

Nota: ogni classe presenta metodi setter e getter per ogni attributo.

2.3 Associazioni e Cardinalità

- **Generalizzazione Animale:** Animale si generalizza in Orso e Pinguino. Si tratta di una generalizzazione totale (tutti gli animali appartengono ad una delle categorie) ed esclusiva (un animale non può essere contemporaneamente un orso e un pinguino).
- **Generalizzazione Item:** Item si generalizza in Cibo e Vestito. Si tratta di una generalizzazione totale (tutti gli item appartengono ad una delle categorie) ed esclusiva (un item non può essere contemporaneamente un cibo e un vestito).
- **Utente-Animale (1 a 0..2):** un utente può possedere un massimo di due animali e ogni animale appartiene a un solo utente.
- **Utente-Item (1 a 0..*):** un utente può acquistare più items.
- **Animale-Vestito (1 a 0..2):** ogni animale può indossare al massimo due vestiti e ogni vestito può essere indossato da un solo animale alla volta.
- **Animale-Minigame (1 a 0..*):** ogni animale può giocare più volte ai minigames.
- **Negozi-Item (1 a 1..*):** il negozio possiede almeno un item acquistabile e ogni item appartiene allo stesso negozio.

2.4 Regole di business

- Un Utente potrà acquistare un Item solo se l'Animale corrente ha $\text{punti} \geq$ al costo di quell'Item.
- Un Utente potrà far giocare l'Animale corrente a un Minigame solo se il suo attributo energia è $\geq$ all'energiaConsumata di quel Minigame.
- Gli animali di uno stesso utente non devono avere lo stesso nome.
- Gli attributi login e password di Utente, e l'attributo nome dell'Animale, sono stringhe con lunghezza non superiore ai 15 caratteri.
- Gli attributi energia e fame dell'Animale possono assumere valori compresi fra 0 e, rispettivamente, l'attributo energiaMax e fameMax .
- Un Utente non può possedere più di 2 animali contemporaneamente.
- Un Animale non può indossare più di 2 istanze di vestiti contemporaneamente.
- Non si può utilizzare un'Item/rimuovere un vestito a un Animale che sta dormendo ($\text{dorme} = \text{true}$).
- Un Animale non può giocare a un Minigame se sta dormendo ($\text{dorme} = \text{true}$).
- Un Vestito indossato non può essere eliminato.
- Un Vestito può essere indossato da un solo animale alla volta.

3. LISTA DELLE FUNZIONI AD ALTO LIVELLO

3.1 Gestione Utente

recuperaID:

- **Input:** nome utente (*String*)
- **Output:** idUtente (*int*)
- **Descrizione:** restituisce l'id numerico associato all'Utente nel momento della registrazione. Invia una query al database che recupera esattamente la riga associata a un certo nome utente (essendo univoco si avrà sempre una sola corrispondenza).

salvaUtente:

- **Input:** nome utente (*String*), password (*String*)
- **Output:** nessuno
- **Descrizione:** inserisce una nuova riga Utente nel database, dotata di nome utente, password e un id numerico generato automaticamente da PostgreSQL. Questo metodo verrà chiamato durante la creazione di un account.

controlloLogin:

- **Input:** nome utente (*String*)
- **Output:** esito (*boolean*)
- **Descrizione:** permette di controllare se nel database è già presente un Utente con lo stesso nome inserito in input. Effettua una query di **SELECT** che cerca un'eventuale riga associata al nome utente, restituendo "true" in caso di riscontro o "false" in caso contrario. Ciò garantisce in primis l'unicità del nome utente.

aggiornaPassword:

- **Input:** idUtente (*int*), nuova password (*String*)
- **Output:** nessuno
- **Descrizione:** effettua una query di **UPDATE** della colonna "password" associata alla chiave primaria idUtente.

cercaUtente:

- **Input:** nome utente (*String*), password (*String*)
- **Output:** esito (*boolean*)
- **Descrizione:** il metodo cerca la presenza nel database di una riga che abbia nome utente e password corrispondenti a quelle inserite. In caso di riscontro viene restituito "true", in caso contrario "false". Si tratta di un controllo necessario nel momento in cui un Utente tenta l'accesso.

3.2 Gestione Animale

recuperaId:

- **Input:** idUtente (*int*), nome animale (*String*)
- **Output:** idAnimale (*int*)
- **Descrizione:** recupera la chiave primaria associata all'animale posseduto dall'utente in input che ha come nome quello inserito (come specificato, gli animali di uno stesso utente hanno nome unico, perciò si avrà sempre solo una corrispondenza).

salvaAnimale:

- **Input:** animale (*Animale*, entità del model), idUtente (*int*)
- **Output:** esito (*boolean*)
- **Descrizione:** inserisce un nuovo animale associato all'utente in input: passando l'oggetto, il metodo prima recupera tutte le singole informazioni dell'animale (nome, fame, energia, fameMax, energiaMax, punti, dorme e specie) per poi passarle alla query.

recuperaListaAnimali:

- **Input:** idUtente (*int*)
- **Output:** lista animali (*ArrayList*)
- **Descrizione:** effettua una query di **SELECT** per recuperare tutti gli animali associati a un certo utente. Ogni animale trovato verrà creato con le relative informazioni nel model e aggiunto all'*ArrayList* di ritorno.

modificaNome:

- **Input:** idAnimale (*int*), nuovo nome (*String*)
- **Output:** nessuno
- **Descrizione:** effettua una query di **UPDATE** della colonna “nome” associata alla chiave primaria idAnimale.

eliminaAnimale:

- **Input:** idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di **DELETE** della riga associata alla chiave primaria idAnimale.

aggiornaStatoAnimale:

- **Input:** animale (*Animale*, entità del model), idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** recupera tutte le singole informazioni dell'animale in input e poi usa questi dati per aggiornare tutte le colonne relative all'id di quell'animale nel database.

resetStatoSonno:

- **Input:** idUtente (*int*)
- **Output:** nessuno
- **Descrizione:** aggiorna la colonna “dorme” di tutti gli animali dell’Utente a *false*.
Nota: Solitamente il programma in chiusura farà in modo che l’attributo **dorme** dell’animale venga salvato sempre come “false” (per necessità di sistema, all’avvio gli animali dovranno essere tutti svegli). Naturalmente, per errori di connessione al database o arresti anomali dell’applicazione, può capitare che lo stato non venga registrato correttamente. All’avvio successivo della sessione utente, questo metodo viene invocato come *failsafe* preventivo per resettare lo stato di sonno nel database ed allinearli correttamente alla GUI; questo metodo resetStatoSonno assicura all’avvio l’integrità dei dati.

3.3 Gestione Cibo

recuperaListaCibo:

- **Input:** nessuno
- **Output:** lista cibo (*ArrayList*)
- **Descrizione:** effettua una query di **SELECT** per recuperare tutti i cibi memorizzati nel database. Questi verranno istanziati con le relative informazioni nel model e aggiunti all’*ArrayList* di ritorno. Il metodo permette quindi di ottenere i cibi di default presenti nel negozio.

recuperaInventarioCibo:

- **Input:** idUtente (*int*)
- **Output:** inventario cibo (*ArrayList*)
- **Descrizione:** effettua una query di **SELECT** per recuperare tutti i cibi posseduti dall’Utente in input. Questi verranno istanziati con le relative informazioni nel model e aggiunti all’*ArrayList* di ritorno. Il metodo permette quindi di ottenere i cibi presenti nell’inventario dell’Utente.

aggiungiAinventarioCibo:

- **Input:** idUtente (*int*), cibo (*Cibo*, entità del model)
- **Output:** idIstanza (*int*)
- **Descrizione:** effettua una prima query per recuperare l’id del cibo in input tramite il nome (essendo questo valore univoco si avrà un’unica corrispondenza). Segue una seconda query che aggiunge il nuovo cibo al database con l’idUtente, l’idCibo e l’idIstanza generato in automatico. Quest’ultimo sarà anche restituito in output.

eliminaDaInventario:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di DELETE per rimuovere in modo permanente un cibo consumato/eliminato dall'inventario dell'utente, identificandolo univocamente tramite il suo codice di istanza.

3.4 Gestione Vestito

recuperaListaVestito:

- **Input:** nessuno
- **Output:** lista vestiti (*ArrayList*)
- **Descrizione:** effettua una query di SELECT per recuperare tutti i vestiti memorizzati nel database. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i vestiti di default presenti nel negozio.

recuperaInventarioVestito:

- **Input:** idUtente (*int*)
- **Output:** inventario vestiti (*ArrayList*)
- **Descrizione:** effettua una query di SELECT per recuperare tutti i vestiti posseduti dall'Utente in input. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i vestiti presenti nell'inventario dell'Utente.

aggiungiAinventarioVestito:

- **Input:** idUtente (*int*), vestito (*Vestito*, entità del model)
- **Output:** idIstanza (*int*)
- **Descrizione:** effettua una prima query per recuperare l'id del vestito in input tramite il nome (essendo questo valore univoco si avrà un'unica corrispondenza). Segue una seconda query che aggiunge il nuovo vestito al database con l'idUtente, l'idVestito e l'idIstanza generato in automatico. Quest'ultimo sarà anche restituito in output.

eliminaDaInventario:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di DELETE per rimuovere in modo permanente un vestito rimosso/eliminato dall'inventario dell'utente, identificandolo univocamente tramite il suo codice di istanza.

indossaVestito:

- **Input:** idIstanza (*int*), idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** esegue una query di UPDATE per associare l'id dell'animale alla riga del vestito nell'inventario, impostando la colonna di controllo (es. `indossato_da`) sul valore dell'idAnimale corrente, attivando così i relativi boost di statistiche.

rimuoviVestito:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** esegue una query di UPDATE sulla riga del vestito nell'inventario per impostare il valore dell'animale che lo indossa a *NULL*. Questo scollega il vestito dall'animale attuale e ripristina i valori massimi standard di fame ed energia.

3.5 Gestione Minigame

recuperaMinigame:

- **Input:** nessuno
- **Output:** lista minigame (*ArrayList*)
- **Descrizione:** effettua una query di SELECT sulla tabella dei minigame per estrarre tutti i giochi disponibili di default all'interno del sistema, recuperandone per ciascuno il nome, la ricompensa in punti e il costo energetico richiesto per l'avvio della partita.

4. ARCHITETTURA DEL SOFTWARE E STRUTTURA DEI PACKAGE

4.1 BCE + DAO Pattern

Per mantenere la modularità del codice, il progetto viene diviso in tre livelli logici separati (BCE), usando in più le interfacce DAO per i collegamenti a livello di database:

- **Package gui (Boundary):** Gestisce la parte grafica del sistema, quello che l'utente vede a schermo e con la quale interagisce. È scritta usando SwingUI designer (es. Home.java). I pulsanti e i campi di testo non comunicano mai direttamente con il database, ma passano sempre attraverso il controller.
- **Package controller (Control):** Contiene la classe `Controller.java`. È la classe che funge da mediatore all'interno del programma: applica le logiche di business dell'applicazione, coordina il flusso delle operazioni, riceve le richieste della gui, fa i controlli di sicurezza e chiama i DAO per salvare o leggere dati.
- **Package model (Entity):** Contiene le classi effettive dei dati (es. Utente, Animale).
- **Package dao e implementazionePostgresDAO (DAO):** Per non vincolare il programma a un database specifico, vengono definite le interfacce nel package dao (es. UtenteDAO.java) che vengono poi implementate concretamente nel package implementazionePostgresDAO usando codice SQL e connessioni JDBC.
- **Package database:** Contiene la classe `ConnessioneDatabase.java` che gestisce il collegamento fisico al database.

4.2 Singleton Pattern per la Persistenza

Per evitare l'apertura di molteplici connessioni al database (che causano rallentamento del programma e consumo di memoria), abbiamo usato il pattern Singleton nella classe `ConnessioneDatabase`. Questo pattern fa sì che venga creata una sola connessione per tutta l'applicazione, e che venga riutilizzata:

```
public class ConnessioneDatabase {

    private static ConnessioneDatabase instance;
    public Connection connection = null;
    private String nome = "postgres";
    private String password = "password";
    private String url = "jdbc:postgresql://127.0.0.1:5432/
Tamagotchi";
    private String driver = "org.postgresql.Driver";

    private ConnessioneDatabase() throws SQLException {
        try {
            Class.forName(driver);
        }
    }
}
```

```

        connection = DriverManager.getConnection(url, nome,
password);

    } catch (ClassNotFoundException ex) {
        System.out.println("Database Connection Creation
Failed : " + ex.getMessage());
        ex.printStackTrace();
    }
}

public static ConnessioneDatabase getInstance() throws
SQLException {
    if (instance == null) {
        instance = new ConnessioneDatabase();
    } else if (instance.connection.isClosed()) {
        instance = new ConnessioneDatabase();
    }
    return instance;
}
}

```

4.3 Incapsulamento e Disaccoppiamento

Il sistema applica l'incapsulamento impedendo la manipolazione diretta dei dati del database (come query SQL, connessione, driver JDBC o file). I DAO incapsulano le query SQL (PreparedStatement), impedendo vulnerabilità di tipo SQL Injection. Il controller mantiene in memoria delle liste specchio degli oggetti correntemente attivi (es. listaAnimali, listaItem) sincronizzandola periodicamente con il DB, riducendo le letture ridondanti. In questo modo il disaccoppiamento fa in modo che le varie parti del programma (come la GUI e il DAO) dipendano il meno possibile l'una dall'altra.

5. GESTIONE DELLE ECCEZIONI

5.1 Gestione su più livelli

Per fare in modo che l'applicazione risulti sicura e robusta, le eccezioni vengono gestite dividendo i ruoli tra i vari package. Quando avviene un errore nel database (es. mancanza del collegamento o inserimento di un dato errato), il codice JDBC solleva una `SQLException`.

1. Il package di implementazione `PostgresDAO` propaga l'eccezione marcando il metodo con `throws SQLException`.
2. Il `Controller` cattura questo errore e lancia una nuova eccezione con un messaggio leggibile.
3. La classe della grafica (gui) cattura quest'ultimo messaggio e lo mostra a schermo all'utente tramite la finestra `JOptionPane.showMessageDialog`.

5.2 Eccezioni personalizzate

Per gestire al meglio le eccezioni specifiche di questo sistema, abbiamo creato delle classi di eccezioni nel package `exceptions` per rendere il codice più leggibile e facile da mantenere:

- **ExceptionAnimale (Runtime Exception):** Sollevata se si presenta un errore durante la creazione di un animale.
- **ExceptionEnergia (Runtime Exception):** Sollevata quando l'animale non ha abbastanza energia per giocare a un minigame.
- **ExceptionFame (Runtime Exception):** Sollevata se quando si dà da mangiare ad un animale, la sua fame è già al massimo.
- **ExceptionMinigame (Runtime Exception):** Sollevata se si presenta un errore durante l'avvio di un minigame.
- **ExceptionPassword (Runtime Exception):** Sollevata se l'utente sbaglia o non inserisce la password quando effettua l'accesso.
- **ExceptionPuntiNonSufficienti (Runtime Exception):** Sollevata se l'utente non ha abbastanza punti per comprare un item.
- **ExceptionTroppiAnimali (Runtime Exception):** Sollevata se l'utente possiede già due animali.
- **ExceptionUtente (Runtime Exception):** Sollevata se l'utente sbaglia o non inserisce il login (username) quando effettua l'accesso.
- **ExceptionVestiti (Runtime Exception):** Sollevata se l'animale indossa già due vestiti.

6. SEQUENCE DIAGRAM DELLE OPERAZIONI CRITICHE

In questa sezione vengono descritte due operazioni critiche del sistema, illustrando i rispettivi sequence diagram multi-livello (Controller/GUI, DAO e Model).

6.1 Operazione 1: creaAnimale

Questa operazione permette all'utente loggato di creare un animale, inizializzato con i suoi "stats" di default.

- **Firma:** public void creaAnimale(String tipo, String nome) throws RuntimeException, SQLException
- **Logica di Business:** Il controller si assicura che non ci siano errori che interferiscano con la creazione dell'animale e poi, in base al tipo, definisce l'animale come orso o pinguino. Crea l'oggetto orso/pinguino e lo inserisce nella listaAnimali dell'utente tramite il metodo aggiungiAnimale(), e nel database tramite DAO.
- **OperazioniDB:** Inserimento di record nella tabella Animale tramite AnimaleDAO.salvaAnimale().

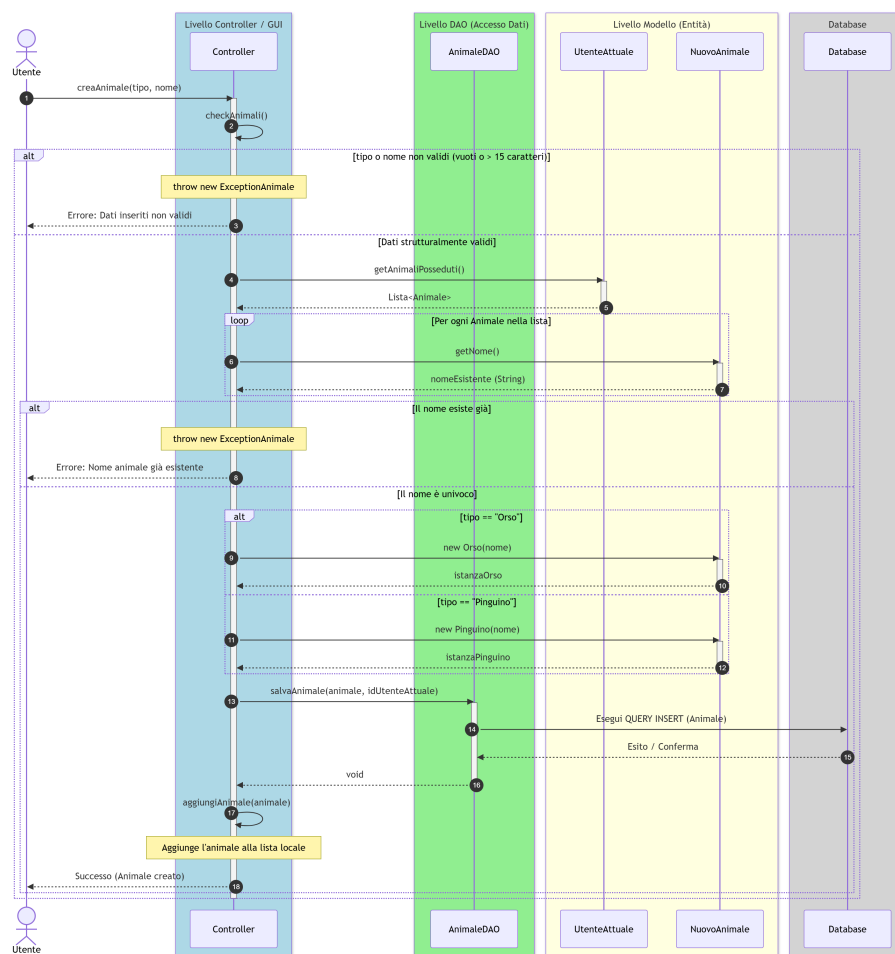


Figura 2: Diagramma di sequenza dell'operazione creaAnimale.

6.2 Operazione 2: compra

- **Firma:** `public void compra(Item item, Animale animale)` throws `RuntimeException`, `SQLException`
- **Logica di Business:** Il controller verifica se l'item è un cibo o un vestito e gli assegna un `idIstanza` in base agli altri item posseduti dall'utente, lo aggiunge alla lista degli item dell'utente e ricalcola i punti dell'animale tramite il metodo `compraItem()`. Salva il risultato di queste operazioni nel database tramite DAO.
- **OperazioniiDB:** Inserimento di record nella tabella `InventarioCibo` o `InventarioVestito` in base al tipo di item tramite `CiboDAO.aggiungiAIventarioCibo()` o `VestitoDAO.aggiungiAIventarioVestito()` rispettivamente.

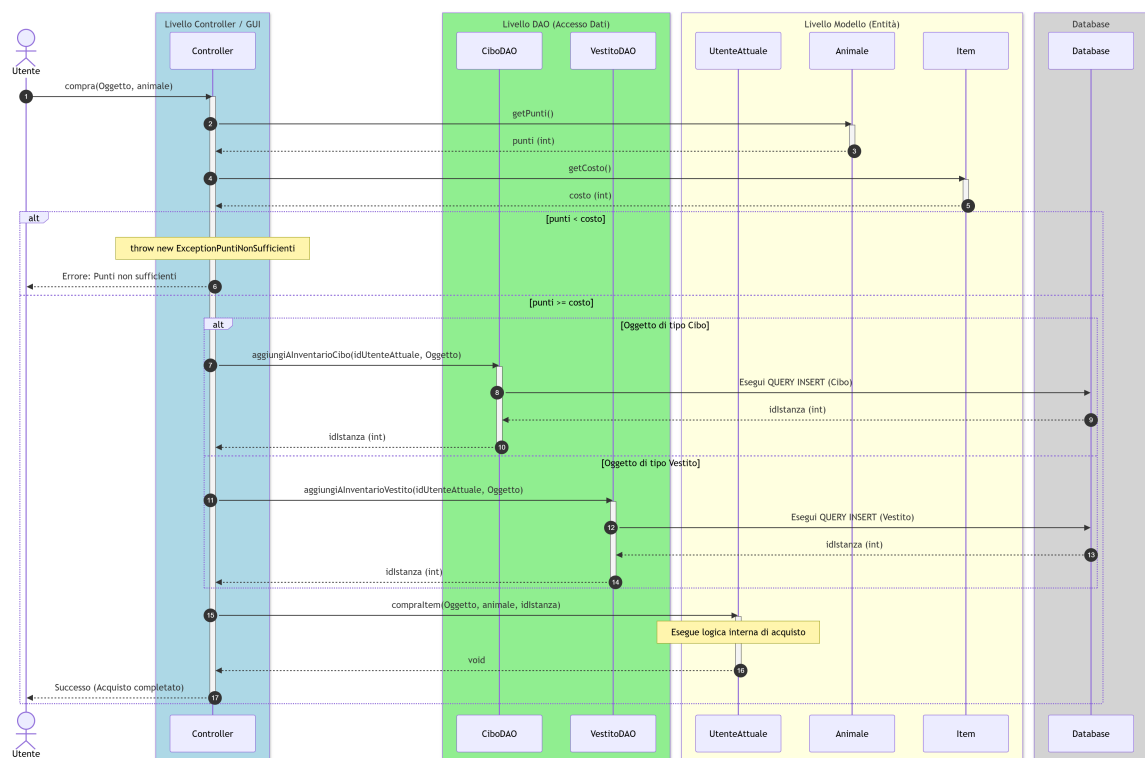


Figura 3: Diagramma di sequenza dell'operazione compra.

7. REPOSITORY GITHUB

- Link repository del progetto:
<https://github.com/raquelzinno/ProgettoPOO/releases/tag/v1.1>